

Principal Component Analysis All in One (v0.8.0-pre)

Version 0.8.0

Zilong Li*

September 29, 2026

Contents

1	Installation	2
1.1	Download compiled binary	2
1.2	Via Conda	2
1.3	Build from source	2
2	Quick start	4
3	Common workflows	5
4	Documentation	5
4.1	Options	5
4.2	Which SVD method to use	7
4.3	Input formats	7
4.4	Output files	7
4.5	Performance and memory	8
4.6	Data Normalization	8
4.7	Projection	9
4.8	Genome-wide selection scan	9
4.9	HWE accounting for population structure	10
4.10	Evaluating the PCA fit (evalAdmix)	10
4.11	Ancestry adjusted LD	10
4.12	Report LD statistics	11
4.13	Pruning based on ancestry adjusted LD	11
4.14	Clumping based on ancestry adjusted LD	11
5	More tutorials	12
5.1	Genotype data (PLINK)	12
5.2	PLINK2 PGEN input	12
5.3	BGEN input (limited support)	12
5.4	Single cell RNA-seq data (CSV)	12
6	Citation	12
7	Contributing and acknowledgements	13

*zilong.dk@gmail.com

PCAone is a fast, memory-efficient C++ tool for principal component analysis of large datasets, with in-core and out-of-core algorithms.

- **Inputs:** PLINK BED, PLINK2 PGEN, BEAGLE genotype likelihoods, zstd-compressed CSV, and limited BGEN support.
- **PCA:** window-based randomized SVD (default), single-pass randomized SVD, IRAM, and exact sample-GRM eigendecomposition.
- **Genetics:** EMU/PCAngsd, projection, selection scans, HWE tests, ancestry-adjusted LD pruning and clumping, and evalAdmix residual correlations.

[Usage guide](#) [Changelog and migration notes](#) [R package](#)

1 Installation

There are 3 ways to install PCAone.

1.1 Download compiled binary

There are compiled binaries provided for both Linux and Mac platform. Check [the releases page](#) to download one.

```
pkg=https://github.com/Zilong-Li/PCAone/releases/latest/download/PCAone-Linux.zip
wget $pkg || curl -LO $pkg
unzip -o PCAone-Linux.zip
```

1.2 Via Conda

PCAone is also available from [bioconda](#).

```
conda config --add channels bioconda
conda install pcaone
PCAone --help
```

1.3 Build from source

PCAone has been tested on both Linux and MacOS system. To build PCAone from the source code, the following dependencies are required:

- GCC/Clang compiler with C++17 support
- GNU make
- zlib

On Linux, we **recommend** building the software from source with MKL as backend to maximize the performance.

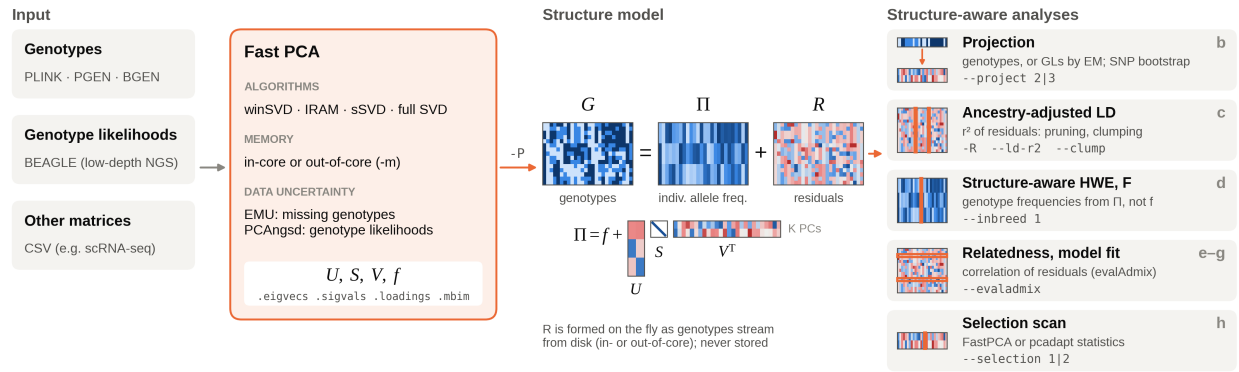
With MKL or OpenBLAS as backend

Build PCAone dynamically with MKL can maximize the performance for large dataset particularly, because the faster threading layer `libiomp5` will be linked at runtime. There are two options to obtain MKL library:

- download MKL from [the website](#)

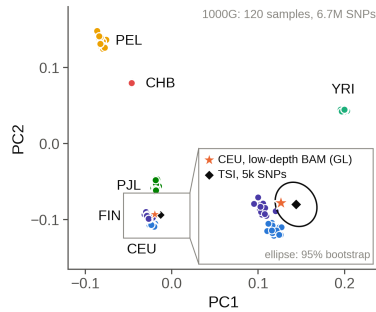
After having MKL installed, find the MKL root path and replace the path below with your own.

a One PCA, reused by every analysis: each test conditions on population structure



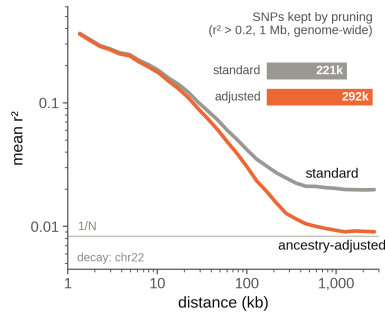
b PCA and projection

-k 5 ; -P ref --project 2|3



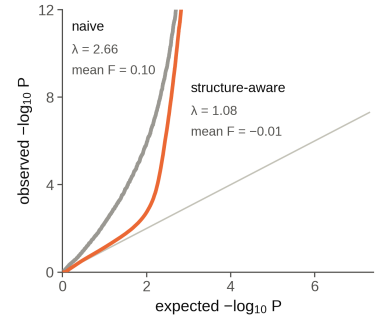
c Ancestry-adjusted LD

-P pcs -R ; -P pcs --ld-r2 0.2



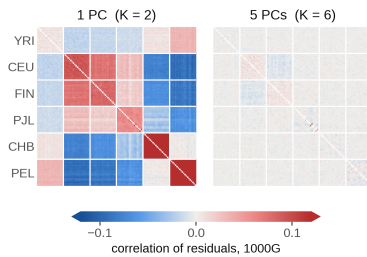
d Structure-aware HWE

-P pcs --inbreed 1



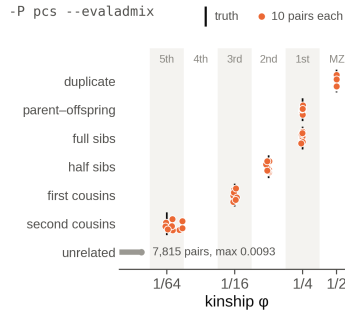
e Model fit

-P pcs -k K-1 --evalAdmix



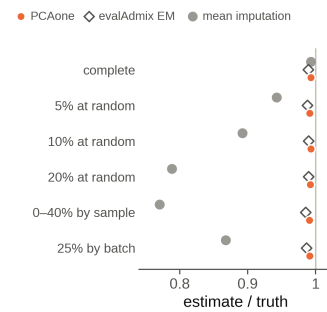
f Relatedness in admixed samples

-P pcs --evalAdmix



g Missing genotypes

PCAngs EM mean imputation



h Genome-wide selection scan (pcadapt)

-P pcs_pruned --selection 2

6.7M SNPs · $\lambda = 1.22$ · PCs from 292k SNPs pruned on adjusted LD · line: Bonferroni · $\blacktriangle$ $-\log_{10} P > 60$

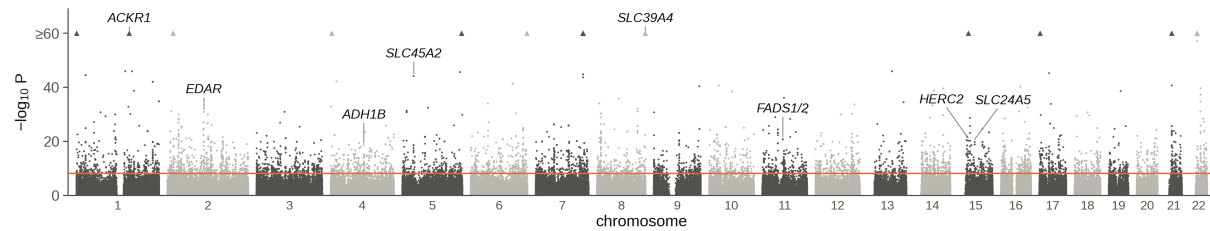


Figure 1: One PCA, reused by structure-aware analyses. (a) PCAone saves U, S, V and the allele frequencies f ; every downstream analysis reads them back with $-P/--USV$. (b–h) Examples on 1000 Genomes samples and on admixed samples with known relatedness.

```
make -j4 MKLROOT=/opt/intel/oneapi/mkl/latest \
  ONEAPI_COMPILER=/opt/intel/oneapi/compiler/latest
```

Alternatively, for advanced user, modify variables directly in **Makefile** and run **make** to use MKL or OpenBlas as backend.

- install MKL by conda

```
conda install -c conda-forge -c anaconda -y mkl mkl-include intel-openmp
git clone https://github.com/Zilong-Li/PCAone.git
cd PCAone
# if mkl is installed by conda then use ${CONDA_PREFIX} as mklroot
make -j4 MKLROOT=${CONDA_PREFIX}
./PCAone -h
```

Without MKL or OpenBLAS dependency

If you don't want any optimized math library as backend, just run:

```
git clone https://github.com/Zilong-Li/PCAone.git
cd PCAone
make -j4
./PCAone -h
```

For MacOS users, check out the [mac workflow](#).

```
brew install libomp
make -j4
```

2 Quick start

```
# PLINK BED/BIM/FAM: compute 10 PCs
PCAone -b data -k 10 -o pcs

# PLINK2 PGEN/PVAR/PSAM: use dosages when available
PCAone -p data -k 10 -o pcs

# Out-of-core PCA with a 2 GB memory setting
PCAone -b data -k 10 -m 2 -o pcs
```

Replace **data** with your input prefix. Add **--hardcall** to use PGEN hard calls. **-m** sizes the working blocks, so total RAM can exceed it. BGEN support is limited; convert to PGEN for production workflows.

Output	Contents
.eigvecs	PC coordinates: samples & PCs
.eigvecs2	The same with FID/IID columns and a header
.eigvals	Eigenvalues
.sigvals	Singular values and scaling metadata
.loadings	Feature loadings, requested with --printv
.mbim	Variant metadata and frequencies with loadings
.log	Run settings and progress

For a small sample count and many variants, use **--svd 3** for exact PCA. It needs an $N \times N$ sample GRM in memory and does not support EM-PCA. Use the default **--svd 2** for large datasets.

3 Common workflows

```
# Ancestry-adjusted LD pruning with the first 3 reference PCs
PCAone -b reference -P ref -k 3 --ld-r2 0.2 -o pruned

# Build a reference and project new samples
PCAone -b reference -k 10 --printv -o ref
PCAone -b target -P ref --project 2 -o projected

# PCA for count data in zstd-compressed CSV
PCAone -c counts.csv.zst -k 10 --scale 2 -S -o counts
```

Projection requires compatible genome builds and alleles. LD requires the same samples in the same order as the reference PCA. Analyses using `-P` use every reference PC unless `-k` selects the leading PCs.

4 Documentation

4.1 Options

Run `PCAone` without arguments to print the common options below, `PCAone --help` to add the advanced ones, or `PCAone --groff > pcaone.1 && man ./pcaone.1` for a man page.

```
General options:
  -h, --help                print all options including hidden advanced options
  -m, --memory arg (=0)    RAM usage in GB unit for out-of-core mode. default is in-core
                           ↪ mode.
                           with --svd 3, it sets the blocks the GRM is streamed in
  -n, --threads arg (=12)  the number of threads to be used
  -v, --verbose arg (=1)   verbosity level for logs. Options are
                           0: silent, no messages on screen;
                           1: concise messages to screen;
                           2: more verbose information;
                           3: enable debug information.

PCA algorithms:
  -d, --svd arg (=2)       SVD method to be applied. default 2 is recommended for big data.
                           ↪ Options are
                           0: the Implicitly Restarted Arnoldi Method (IRAM);
                           1: the Yu's single-pass Randomized SVD with power iterations;
                           2: the accurate window-based Randomized SVD method (PCAone);
                           3: exact PCA by eigendecomposition of the sample GRM, streamed
                           ↪ block by block when N <= M,
                           in N x N memory (no EM-PCA support).
  -k, --pc arg (=10)       top k principal components (PCs) to be calculated. with
                           ↪ -P/--USV, the number
                           of leading PCs of the reference to use (default all)
  -C, --scale arg (=9)     do normalization or scaling for input file. Options are
                           -9: standardize genetic data by sqrt(ploidy*f*(1-f));
                           0: do nothing and proceed to SVD;
                           1: do direct standardization, as the scale(x, center=TRUE,
                           ↪ scale=TRUE) function in R;
                           2: do first count per median log transformation (CPMED), then
                           ↪ standardization;
                           3: do first log1p transformation, then standardization;
                           4: do first relative counts, then standardization.
  --maxp arg (=20)         maximum number of power iterations for RSVD algorithm.
  -S, --no-shuffle         do not shuffle columns of data for --svd 2 (if not locally
                           ↪ correlated).
  --seed arg (=112)        seeds for reproducing results.
  --emu                   use EMU algorithm for genotype input with missingness. not with
                           ↪ --svd 3.
```

```

--pcangsd                                use PCAngsd algorithm for genotype likelihood input. not with
↳ --svd 3.

Input options:
-b, --bfile arg                          prefix of PLINK .bed/.bim/.fam files.
-p, --pgen arg                          prefix of PLINK2 .pgen/.pvar/.psam files.
-c, --csv arg                          path of comma separated CSV file compressed by zstd.
-g, --bgen arg                          path of BGEN file compressed by gzip/zstd.
-G, --beagle arg                        path of BEAGLE file compressed by gzip.
-F, --match-bim arg                    the .mbim file to be matched, where the 7th column is allele
↳ frequency.
-P, --USV arg                          prefix of PCAone .eigvecs/.sigvals/.loadings/.mbim.

Output options:
-o, --out arg (=pcaone)                 prefix of output files. default [pcaone].
-V, --printv                            output the right eigenvectors with suffix .loadings.
-R, --print-r2                          print LD R2 to *.ld.gz file for pairwise SNPs within a window
↳ controlled by --ld-bp.

Misc options:
--maf arg (=0)                         exclude variants with MAF lower than this value. default is 0.05
↳ for

--project arg (=0)                     BEAGLE input, as in PCAngsd, and 0 (no filter) otherwise
project the new samples onto the existing PCs. Options are
0: disabled;
1: by multiplying the loadings with mean imputation for missing
↳ genotypes;
2: by solving the least squares system  $Vx=g$ . skip sites with
↳ missingness;
3: by EM to account for genotype uncertainty (BEAGLE input).
run SNP bootstrap diagnostics for --project 2 using this many

--project-bootstrap arg (=0)           0: disabled;
↳ replicates.
--project-bootstrap-save               1: by multiplying the loadings with mean imputation for missing
↳ *.proj.bootstrap.eigvecs.          2: by solving the least squares system  $Vx=g$ . skip sites with
--inbreed arg (=0)                    3: by EM to account for genotype uncertainty (BEAGLE input).
compute the inbreeding coefficient accounting for population
↳ structure. Options are
0: disabled;
1: compute per-site inbreeding coefficient and HWE test.
compute the correlation of residuals (evalAdmix) given existing

--evaladmix                           (same samples in the same order).
↳ PCs from -P/--USV                 compute selection statistics. Options are
0: disabled;
1: perform selection scan using Galinsky et al method;
2: perform selection scan using PCAdapt method.
R2 cutoff for LD-based pruning (usually 0.2).

--selection arg (=0)                  physical distance threshold in bases for LD window.
statistics to compute LD R2 for pairwise SNPs. Options are
0: the ancestry adjusted, i.e. correlation between the residuals
↳ after
removing the PCs given by -P/--USV (reads .eigvecs of the
↳ same samples);
1: the standard, i.e. correlation between two alleles.

--clump arg                           assoc-like file with target variants and pvalues for clumping.
--clump-names arg (=CHR,BP,P)         column names in assoc-like file for locating chr, pos and
↳ pvalue.
--clump-p1 arg (=0.0001)              significance threshold for index SNPs.
--clump-p2 arg (=0.01)                secondary significance threshold for clumped SNPs.
--clump-r2 arg (=0.5)                 r2 cutoff for LD-based clumping.
--clump-bp arg (=250000)              physical distance threshold in bases for clumping.

```

4.2 Which SVD method to use

This depends on your datasets, particularly the relationship between number of samples (N) and the number of variants / features (M) and the top PCs (k). Here is an overview and the recommendation.

Method	Scenario	Accuracy	Speed
Exact (-d 3)	small N, $N \ll M$	Exact	fastest for N below ~2000
winSVD (-d 2)	M or $N \gg 500000$	Very high	fast (only 7 iterations used)
IRAM (-d 0)	speed insensitive	Very high	depends on N and # iterations
sSVD (-d 1)	accuracy insensitive	High	depends on # iterations

4.3 Input formats

PCAone is designed to be extensible to accept many different formats. Currently, PCAone can work with SNP major genetic formats to study population structure, such as **PLINK**, **BGEN** and **Beagle**. Also, PCAone supports a comma delimited CSV format compressed by zstd, which is useful for other datasets requiring specific normalization such as single cell RNAs data. PCAone also supports **PLINK2 PGEN** input via `--pgen`. If dosages are stored in the `.pgen` file, PCAone uses them by default; add `--hardcall` to force hard-call genotypes instead. The current **BGEN** support is limited, so for large production workflows we recommend converting **BGEN** to **PGEN** when possible.

4.4 Output files

Eigen vectors

Eigen vectors are saved in file with suffix `.eigvecs`. Each row represents a sample and each col represents a PC.

Eigen/Singular values

Eigenvalues and singular values are saved in file with suffix `.eigvals` and `.sigvals` respectively. Each row represents the eigenvalue/singularvalue of corresponding PC.

Features loadings

Features Loadings are saved in file with suffix `.loadings`. Each row represents a feature and each column represents a corresponding PC. Use `--printv` option to output it.

Variant information

A plink-like bim file named with `.mbim` is used to store the variants list with extra information. Currently, the `mbim` file has 7 columns with the 7th being the allele frequency. PCAone writes this file automatically whenever it outputs `.loadings` via `--printv`.

LD R2

The LD R2 for pairwise SNPs within a window can be outputted to a file with suffix `ld.gz` via `--print-r2` option. This file uses the same long format as the one **plink** used.

4.5 Performance and memory

PCAone has both **in-core** and **out-of-core** mode for 3 different partial SVD algorithms, which are IRAM (`--svd 0`), sSVD (`--svd 1`) and winSVD (`--svd 2`). Also, PCAone supports an exact PCA (`--svd 3`). When $N \leq M$, it streams the data block by block and holds only the $N \times N$ GRM (`-m` sets the block size); with `--maf`, or when $N > M$, it runs in-core. Therefore, there are **7** ways for doing PCA in PCAone. In default PCAone uses **in-core** mode, which is the fastest way (**NOTE**: you can gain some speedup for in-code computation by limiting the `-n threads` to half of the available threads of your machine). However, in case the server runs out of memory, you can trigger **out-of-core** mode by specifying the amount of memory using `-m/--memory` option with a value greater than 0. Normally, use `-m 1` is enough for large dataset and PCAone will allocate more RAM when needed.

Run winSVD method (default) with in-core mode

```
./PCAone --bfile example/plink
```

Run winSVD method with out-of-core mode

```
./PCAone --bfile example/plink -m 2
```

Out-of-core BED winSVD writes a shuffled copy of the BED before the PCA. It assigns the SNPs at random (`--seed`) to the `-w` bands of read blocks and keeps their source order within each band. winSVD only updates its test matrix between bands, so the PCs equal those of a full random permutation in exact arithmetic. Floating-point rounding can differ slightly. The rewrite reads the BED once and writes it once, in large writes whatever its size. It needs disk space for one more BED. Samples keep their order; `-S` turns the shuffle off.

`--buffer` (default 2 GiB) bounds the two genotype buffers of the rewrite; it does not change the order. SNP indices and the BIM lines take additional memory. The permuted BED/BIM/FAM (`<out>.perm.*`) are removed after the PCA unless `-v 3` is used, and `-o` may not point them at the input.

Run sSVD method with out-of-core mode

```
./PCAone --bfile example/plink --svd 1 -m 2
```

Run IRAM method with out-of-core mode

```
./PCAone --bfile example/plink --svd 0 -m 2
```

Run the exact PCA (streams the GRM when $N \leq M$)

```
./PCAone --bfile example/plink --svd 3
```

4.6 Data Normalization

Use `--scale` to choose normalization. Genetic data are centred and standardized by allele frequency by default. CSV input is left as it is (neither centred nor scaled) with the default and `--scale 0`; modes 14 standardize, after the optional count or log transformation described in the options above.

4.7 Projection

Project new samples onto existing PCs is supported with `--project` option. First, we run PCAone on a set of reference samples and output the loadings:

```
PCAone -b example/ref -k 10 --printv -o ref
```

Pass the reference prefix with `-P/--USV` to read its loadings, singular values and allele frequencies. PCAone matches overlapping markers against `.mbim` and corrects flipped alleles. Both datasets must use the same genome build and compatible alleles.

Below is a command using example PLINK data to project new target samples onto the reference coordinates.

```
## --USV: prefix to .eigvecs, .sigvals, .loadings, .mbim
## --project: check the manual on projection methods
PCAone -b example/new \
  --USV ref \
  --project 2 \
  -o new
```

The projection uses every PC in the reference; add `-k` to use only the leading ones (e.g. `-k 3`). This holds for every analysis that reads a reference with `-P/--USV: --project, --selection, --inbreed, --evaladmix` and the LD analyses below, so one reference run with a generous `-k` serves them all.

To visualize uncertainty in projected coordinates, use `--project-bootstrap` to resample matched SNPs with replacement after the baseline `--project 2` projection. PCAone writes per-sample/per-PC summaries to `.proj.bootstrap.tsv` and PC-pair covariance matrices to `.proj.bootstrap.cov.tsv`, which can be used to draw uncertainty ellipses around the baseline estimates. Add `--project-bootstrap-save` to write raw bootstrap replicate coordinates to `.proj.bootstrap.eigvecs`.

```
PCAone -b example/new \
  --USV ref \
  --project 2 \
  --project-bootstrap 100 \
  -o new
```

For BEAGLE genotype likelihood input, use `--project 3` to perform genotype-likelihood aware projection with an EM procedure:

```
PCAone -G example/target.beagle.gz \
  --USV ref \
  --project 3 \
  -o target
```

The target is scaled as the reference PCA was (recorded in its `.sigvals`), so `--scale` is not needed here.

NB: The two BEAGLE alleles must be the two alleles of the reference `.mbim`, in either order. BEAGLE likelihoods count `allele2`, which PCAone matches to the reference's counted allele `A1` (5th column of `.mbim`), and sites where they are swapped are flipped automatically; sites with other alleles are dropped. Use `angsd -doMajorMinor 3` with a `-sites` file taken from the reference `.mbim` to fix the alleles.

4.8 Genome-wide selection scan

PCAone can test selection for differentiated variants after a good reference PCA has been computed with good sites after site-level QC, e.g. LD pruning.

First run PCA on the subset genotype matrix with good sites

```
PCAone -b example/subset -k 10 -o goodsites
```

Then reuse the saved USV outputs with `--selection`. Method 1 computes the Galinsky/FastPCA statistic for each variant and PC, while method 2 computes pcadapt-style z-scores, chi-square statistics, p-values, and a genomic inflation factor.

```
PCAone -b example/fullset \  
--USV goodsites \  
--selection 1 \  
-o sel
```

```
PCAone -b example/fullset \  
--USV goodsites \  
--selection 2 \  
-o sel
```

The selection outputs are written per variant in the same marker order as the plink `.bim` file.

4.9 HWE accounting for population structure

To test Hardy-Weinberg equilibrium in presence of population structure like `pcangsd`, we need to work on the so-called individual allele frequencies matrix Π , which can be reconstructed via the output of PCAone, i.e the `.eigvecs`, `.sigvals`, `.loadings` and `.mbim` files, generated by first running

```
PCAone -b example/plink -k 3 -V -o pcaone --pcangsd  
## alternative for PLINK input with missingness  
## PCAone -b example/plink -k 3 -V -o pcaone --emu
```

Then we apply `--inbreed 1` option to obtain the P value of HWE and inbreeding coefficient per-site. The per-site statistics is outputted to a file with suffix `.hwe`.

```
PCAone -b example/plink \  
--USV pcaone \  
--inbreed 1 \  
-o inbreed
```

4.10 Evaluating the PCA fit (evalAdmix)

`--evaladmix` computes the $N \times N$ correlation of residuals after predicting each genotype from the PCs of a previous run of the same samples, in the same order. Near-zero entries mean the PCs capture the structure; for related samples the entry estimates twice their kinship. PCAone writes `.corres` and `.kinship` (`.corres` divided by 2). Use $K-1$ PCs for K ancestral groups and apply the same `--maf` in both runs.

```
PCAone -b example/plink -k 3 --maf 0.05 -o pcs  
PCAone -b example/plink -P pcs --evaladmix --maf 0.05 -o eval
```

See [the evalAdmix guide](#) for the method and benchmarks.

4.11 Ancestry adjusted LD

LD patterns vary across diverse ancestry and structured groups, and conventional LD statistics, e.g. the implementation in `plink --ld`, failed to model the LD in admixed populations. Thus, we can use the so-called ancestry-adjusted LD statistics to account for population structure in LD. See our [paper](#) for more details.

The ancestry-adjusted LD is the correlation between the residuals of the genotypes after removing the top principal components, which capture the population structure. We first figure out the number of PCs (`-k/--pc`) that capture population structure and run the PCA. In this example, assuming that 3 PCs can account for population structure:

```
./PCAone -b example/plink -k 3 -o adj
```

Then pass its prefix to `-P/--USV` in any of the LD analyses below. PCAone reads the genotypes again, removes the PCs in `adj.eigvecs` from them as they are read, and computes the LD from the residuals, so no residual matrix is written to disk. The PCs must come from the same samples in the same order, but the variants need not be the ones the PCA used (e.g. a different `--maf`). This works in-core and out-of-core (`-m`), for PLINK (`-b`) and PGEN (`-p`) input. By default every PC in `.eigvecs` is removed; `-k/--pc` removes only the leading ones, so one reference PCA run with e.g. `-k 10` serves any smaller number of PCs (`-P adj -k 3`). Use `--ld-stats 1` without `-P` for the standard LD instead. See [the LD methods guide](#) for the derivation and implementation.

4.12 Report LD statistics

Currently, the LD R2 for pairwise SNPs within a window can be outputted via `--print-r2` option.

```
./PCAone -b example/plink \  
-P adj \  
--ld-bp 1000000 \  
--print-r2 \  
-o adj
```

We provide the `calc_decay_bin.R` script to parse the output file `.ld.gz` and calculate the average R2 for each bin as well as plotting. We also provide the nextflow `ld.nf` for benchmarking the LD statistics.

4.13 Pruning based on ancestry adjusted LD

Given the PCs, we can do pruning based on user-defined thresholds and windows. Of each pair of variants above the R2 cutoff, the one with the lower MAF is removed. The kept and removed variants are written to `.ld.prune.in` and `.ld.prune.out`.

```
./PCAone -b example/plink \  
-P adj \  
--ld-r2 0.8 \  
--ld-bp 1000000 \  
-o adj
```

4.14 Clumping based on ancestry adjusted LD

Likewise, we can do clumping based on the ancestry-adjusted LD and user-defined association results

```
./PCAone -b example/plink \  
-P adj \  
--clump example/plink.pheno0.assoc,example/plink.pheno1.assoc \  
--clump-p1 0.01 \  
--clump-p2 0.05 \  
--clump-r2 0.1 \  
--clump-bp 10000000 \  
-o adj
```

5 More tutorials

Let's download the example data first if you haven't done so.

```
wget http://popgen.dk/zilong/datahub/pca/example.tar.gz
tar -xzf example.tar.gz && rm -f example.tar.gz
```

5.1 Genotype data (PLINK)

Run exact PCA with `--svd 3` and plot the sample coordinates in R.

```
./PCAone --bfile example/plink -d 3
```

Then, we can make a PCA plot in R.

```
pcs <- read.table("pcaone.eigvecs2",h=F)
plot(pcs[,3:4], col=factor(pcs[,1]), xlab="PC1", ylab="PC2", cex.lab=1.5)
legend("topleft", legend=levels(factor(pcs[,1])), col=1:4, pch = 21, cex=1.2)
```

5.2 PLINK2 PGEN input

PCAone can also read PLINK2 `.pgen/.pvar/.psam` input directly. By default it uses dosages when they are present in the `.pgen` file.

```
./PCAone --pgen example/plink2 -k 10 -m 2
```

If you want to ignore dosages and use hard-call genotypes instead, add `--hardcall`.

```
./PCAone --pgen example/plink2 -k 10 -m 2 --hardcall
```

5.3 BGEN input (limited support)

NB: BGEN support is very limited. Please convert BGEN to PGEN and use PGEN input!

Imputation tools usually generate the genotype probabilities or dosages in BGEN format. To do PCA with the imputed genotype probabilities, we can work on BGEN file with `--bgen` option instead.

```
./PCAone --bgen example/test.bgen -k 10 -m 2
```

5.4 Single cell RNA-seq data (CSV)

In this example, we run PCA for the scRNAs-seq data using CSV format with a normalization method called count per median log transformation (CPMED). Since the features (genes) tend to be not correlated locally, we use `-S` option to disable permutation for winSVD.

```
./PCAone --csv example/BrainSpinalCord.csv.zst -k 10 -m 2 --scale 2 -S
```

6 Citation

Please cite [Fast and accurate out-of-core PCA framework for large scale biobank data](#) when using PCAone. For specific analyses, also cite:

- EMU: [Large-scale inference of population structure in presence of missingness using PCA.](#)
- PCAngsd: [Inferring Population Structure and Admixture Proportions in Low-Depth NGS Data.](#)

- Ancestry-adjusted LD: [Measuring linkage disequilibrium and improvement of pruning and clumping in structured populations](#).

7 Contributing and acknowledgements

Issues and contributions are welcome. PCAone uses [Eigen](#), [Spectra](#), and a BGEN reader adapted from [jeremymcrae/bgen](#); its EMU and PCAngsd implementations build on [Jonas Meisner's packages](#).